Assignment No.1

Title: Design a program to maintain a student database that performs the following tasks
1. Add and store student details (ID, Name, CGPA) using dynamically allocated memory.
2. Expand the student list using realloc() as new entries are added.
3. Implement Linear Search and Binary Search to find student records by ID.
4. Implement at least two Sorting Algorithms (Bubble Sort, Selection Sort, or Insertion Sort) to sort student records by: Name (Alphabetically),CGPA (Ascending/Descending)

```
#include <iostream>
#include <cstdlib>   // For malloc(), realloc(), free()
#include <cstring>   // For strcmp
using namespace std;

struct Student
 {
   int id;
   char name[50];
   float cgpa;
};

// Function declarations

void addStudent(Student*  &students, int &count, int &capacity);
void displayStudents(Student* students, int count);
int linearSearch(Student* students, int count, int keyID);
int binarySearch(Student* students, int count, int keyID);
void sortByID(Student* students, int count); // Required before binary search

int main()
{
   Student* students = nullptr;
   int count = 0, capacity = 0;
   int choice, searchID;

   do
{
      cout << "\n--- Student Database Menu ---\n";
      cout << "1. Add Student\n";
      cout << "2. Display Students\n";
      cout << "3. Linear Search by ID\n";
      cout << "4. Binary Search by ID\n";
      cout << "5. Exit\n";
      cout << "Enter your choice: ";
      cin >> choice;
```

```
  switch(choice)
{
        case 1:
            addStudent(students, count, capacity);
            break;
        case 2:
            displayStudents(students, count);
            break;
        case 3:
            cout << "Enter ID to search: ";
            cin >> searchID;
            {
                int index = linearSearch(students, count, searchID);
                if (index != -1)
                    cout << "Found: " << students[index].name << ", CGPA: " << students[index].cgpa << "\n";
                else
                    cout << "Student not found.\n";
            }
            break;
        case 4:
            sortByID(students, count);                          // Binary search needs sorted list
            cout << "Enter ID to search: ";
            cin >> searchID;
            {
                int index = binarySearch(students, count, searchID);
                if (index != -1)
                    cout << "Found: " << students[index].name << ", CGPA: " << students[index].cgpa << "\n";
                else
                    cout << "Student not found.\n";
            }
            break;
        case 5:
            break;
        default:
            cout << "Invalid choice!\n";
      }

   } while (choice != 5);

   free(students);                                          // Free dynamically allocated memory
   return 0;
}
```

```
// Add a student and expand list using realloc()

void addStudent(Student*& students, int& count, int& capacity)
{
    if (count == capacity)
    {
        capacity += 1;
        students = (Student*) realloc(students, capacity * sizeof(Student));
        if (!students)
        {
            cout << "Memory allocation failed!\n";
            exit(1);
        }
    }

    cout << "Enter Student ID: ";
    cin >> students[count].id;
    cin.ignore();
    cout << "Enter Student Name: ";
    cin.getline(students[count].name, 50);
    cout << "Enter CGPA: ";
    cin >> students[count].cgpa;
    count++;
}


// Display all student records

void displayStudents(Student* students, int count)
{
    if (count == 0)
    {
        cout << "No student records available.\n";
        return;
    }

    cout << "\n--- Student List ---\n";
    for (int i = 0; i < count; i++)
    {
        cout << "ID: " << students[i].id
             << ", Name: " << students[i].name
             << ", CGPA: " << students[i].cgpa << "\n";
    }
}
```

```
// Linear search by ID

int linearSearch(Student* students, int count, int keyID)
{
    for (int i = 0; i < count; i++)
    {
        if (students[i].id == keyID)
            return i;
    }
    return -1;
}

// Binary search by ID (requires sorted array)

int binarySearch(Student* students, int count, int keyID)
{
    int low = 0, high = count - 1;
    while (low <= high)
    {
        int mid = (low + high) / 2;
        if (students[mid].id == keyID)
            return mid;
        else if (students[mid].id < keyID)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

// Sort the list by ID (ascending) for binary search

void sortByID(Student* students, int count)
{
    for (int i = 0; i < count - 1; i++)
       {
        for (int j = 0; j < count - i - 1; j++)
           {
            if (students[j].id > students[j + 1].id)
            {
                swap(students[j], students[j + 1]);
            }
        }
    }
}
```

//OUTPUT

1. Add Student
> ID: 101
> Name: Rahul
> CGPA: 8.2

3. Linear Search by ID
> Enter ID: 101
> Found: Rahul, CGPA: 8.2

4. Binary Search by ID
> Enter ID: 101
> Found: Rahul, CGPA: 8.2

Assignment No.2

Title: Design a program to maintain a student database that performs the following tasks
1. Add and store student details (ID, Name, CGPA) using dynamically allocated memory.
2. Expand the student list using realloc() as new entries are added.
3. Implement at least two Sorting Algorithms (Bubble Sort, Selection Sort, or Insertion Sort) to sort student records by, Name (Alphabetically), CGPA (Ascending/Descending)

```
#include <iostream>
#include <cstdlib>
#include <cstring>
using namespace std;

struct Student {
    int id;
    char name[50];
    float cgpa;
};

// Function declarations
void addStudent(Student*& students, int& count, int& capacity);
void displayStudents(Student* students, int count);
void bubbleSortByName(Student* students, int count);
void selectionSortByCGPA(Student* students, int count, bool ascending);

int main() {
    Student* students = nullptr;
    int count = 0, capacity = 0;
    int choice;

    do {
        cout << "\n--- Student Database Menu ---\n";
        cout << "1. Add Student\n";
        cout << "2. Display Students\n";
        cout << "3. Sort by Name (Bubble Sort)\n";
        cout << "4. Sort by CGPA (Selection Sort)\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch(choice) {
            case 1:
                addStudent(students, count, capacity);
                break;
            case 2:
                displayStudents(students, count);
```

```
            break;
        case 3:
            bubbleSortByName(students, count);
            cout << "Students sorted by name (alphabetically).\n";
            break;
        case 4: {
            char order;
            cout << "Enter 'a' for ascending or 'd' for descending CGPA: ";
            cin >> order;
            selectionSortByCGPA(students, count, order == 'a' || order == 'A');
            break;
        }
        case 5:
            break;
        default:
            cout << "Invalid choice!\n";
    }

    } while (choice != 5);

    free(students);
    return 0;
}

// Add a student
void addStudent(Student*& students, int& count, int& capacity) {
    if (count == capacity) {
        capacity += 1;
        students = (Student*) realloc(students, capacity * sizeof(Student));
        if (!students) {
            cout << "Memory allocation failed!\n";
            exit(1);
        }
    }

    cout << "Enter Student ID: ";
    cin >> students[count].id;
    cin.ignore();
    cout << "Enter Student Name: ";
    cin.getline(students[count].name, 50);
    cout << "Enter CGPA: ";
    cin >> students[count].cgpa;
    count++;
}

// Display all students
```

```
void displayStudents(Student* students, int count) {
    if (count == 0) {
        cout << "No student records available.\n";
        return;
    }

    cout << "\n--- Student List ---\n";
    for (int i = 0; i < count; i++) {
        cout << "ID: " << students[i].id
             << ", Name: " << students[i].name
             << ", CGPA: " << students[i].cgpa << "\n";
    }
}

// Bubble Sort by Name (Alphabetical)
void bubbleSortByName(Student* students, int count) {
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (strcmp(students[j].name, students[j + 1].name) > 0) {
                swap(students[j], students[j + 1]);
            }
        }
    }
}

// Selection Sort by CGPA
void selectionSortByCGPA(Student* students, int count, bool ascending) {
    for (int i = 0; i < count - 1; i++) {
        int minOrMax = i;
        for (int j = i + 1; j < count; j++) {
            if ((ascending && students[j].cgpa < students[minOrMax].cgpa) ||
                (!ascending && students[j].cgpa > students[minOrMax].cgpa)) {
                minOrMax = j;
            }
        }
        if (minOrMax != i)
            swap(students[i], students[minOrMax]);
    }

    cout << "Students sorted by CGPA " << (ascending ? "(Ascending)." : "(Descending).") << "\n";
}
```

Assignment No.3

Title: Write a program to Implement stack as a ADT and perform following operation 1. Infix to postfix expression.

```
#include<iostream>
#include<cstring>
#include<math.h>
using namespace std;
class stack
{
private :
        typedef struct stack1
        {
        int data;
        struct stack1 *next;
        }node;
        node *top;
public:
stack();
        ~stack();
        void push(char,node **);
        int pop(node **);
        int stackempty(node *);
void postfix();
        };

int main()
{
int ch;
stack st;
do
{
cout<<"\n1.Infix to postfix \n2. Exit";
cout<<"\n Enter your choice";
cin>>ch;
switch(ch)
{
case 1: st.postfix();
       break;
case 2: break;
}
}while(ch!=2);
}
```

```
stack::stack()
{
top=NULL;
}

stack :: ~stack()
{
node *temp;
temp=top;
if(temp==NULL)
delete temp;
else
{
while(temp!=NULL)
{
temp=temp->next;
top=NULL;
top=temp;
}
delete temp;
}
}

void stack :: push(char x,node **top)
{
node *newnode;
newnode=new node;
newnode->next=NULL;
newnode->data=x;
newnode->next=*top;
*top=newnode;
}

int stack::pop(node **top)
{
int x;
node *temp;
x=(*top)->data;
temp=*top;
*top=(*top)->next;
delete temp;
return x;
}
```

```
int stack :: stackempty(node *temp)
{
if(temp==NULL)
return 1;
else
return 0;
}

void stack::postfix()
{
char exp[40];
int length,j,l;
char ch;
char post[40];
j=0;
cout<<"\n enter exp";
cin>>exp;
length=strlen(exp);
cout<<"\n\n The postfix is:";
for(int i=0;i<=length-1;i++)
{
ch=exp[i];
if(ch=='('||ch=='+'||ch=='-'||ch=='*'||ch=='/')
push(ch,&top);
else if(ch!=')')
{
post[j]=ch;
j++;
cout<<ch;
}
else
{
do
{
ch=pop(&top);
if(ch!='(')
{
post[j]=ch;
j++;
cout<<ch;
}
}while(ch!='(');
}
}
post[j]='$';
}
```

Output:
1. Infix to postfix
2. Exit
Enter your choice 1
Enter Infix Exp: (a+b)
Postfix Exp is:ab+

Assignment No.4

Title: Program to implement Circular Queue.

```
#include <iostream>
using namespace std;
int cqueue[5];
int front = -1, rear = -1, n=5;
void insertCQ(int val) {
  if ((front == 0 && rear == n-1) || (front == rear+1)) {
    cout<<"Queue Overflow \n";
    return;
  }
  if (front == -1) {
    front = 0;
    rear = 0;
  } else {
    if (rear == n - 1)
    rear = 0;
    else
    rear = rear + 1;
  }
  cqueue[rear] = val ;
}
void deleteCQ() {
  if (front == -1) {
    cout<<"Queue Underflow\n";
    return ;
  }
  cout<<"Element deleted from queue is : "<<cqueue[front]<<endl;

  if (front == rear) {
    front = -1;
    rear = -1;
  } else {
    if (front == n - 1)
    front = 0;
    else
    front = front + 1;
  }
}
void displayCQ() {
  int f = front, r = rear;
  if (front == -1) {
    cout<<"Queue is empty"<<endl;
    return;
  }
```

```
  cout<<"Queue elements are :\n";
  if (f <= r) {
    while (f <= r){
      cout<<cqueue[f]<<" ";
      f++;
    }
  } else {
    while (f <= n - 1) {
      cout<<cqueue[f]<<" ";
      f++;
    }
    f = 0;
    while (f <= r) {
      cout<<cqueue[f]<<" ";
      f++;
    }
  }
  cout<<endl;
}
int main()
{
  int ch, val;
  cout<<"1)Insert\n";
  cout<<"2)Delete\n";
  cout<<"3)Display\n";
  cout<<"4)Exit\n";
  do {
    cout<<"Enter choice : "<<endl;
    cin>>ch;
    switch(ch) {
      case 1:
      cout<<"Input for insertion: "<<endl;
      cin>>val;
      insertCQ(val);
      break;
      case 2:
      deleteCQ();
      break;
      case 3:
      displayCQ();
      break;
      case 4:
      cout<<"Exit\n";
      break;
      default: cout<<"Incorrect!\n";
    }
```

```
  } while(ch != 4);
  return 0;
}
```

Output

```
1)Insert
2)Delete
3)Display
4)Exit
Enter choice : 1
Input for insertion:
Enter choice : 1
Input for insertion:
Enter choice : 1
Input for insertion:
Enter choice : 1
Input for insertion:
Enter choice : 1
Input for insertion:
Enter choice : 2
Element deleted from queue is : 5
Enter choice : 2
Element deleted from queue is : 3
Enter choice : 2
Element deleted from queue is : 2
Enter choice : 1
Input for insertion: 6
Enter choice : 3
Queue elements are :
7 9 6
Enter choice : 4
Exit
```

Assignment No.5

Title: Implement binary search tree and perform following operations:

a) Insert (Handle insertion of duplicate entry)
b) Delete
c) Search
d) Display tree (Traversal)
e) Display - Depth of tree
f) Display - Mirror image
g) Create a copy
h) Display all parent nodes with their child nodes
i) Display leaf nodes
j) Display tree level wise

```
from collections import deque
import copy

class BSTNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None

    # Insert (rejects duplicates)

    def insert(self, key):
        def _insert(node, key):
            if node is None:
                return BSTNode(key)
            if key < node.key:
                node.left = _insert(node.left, key)
            elif key > node.key:
                node.right = _insert(node.right, key)
            else:
                print(f"Duplicate entry '{key}' not inserted.")
            return node
        self.root = _insert(self.root, key)
```

```
# Search

def search(self, key):
    node = self.root
    while node:
        if key < node.key:
            node = node.left
        elif key > node.key:
            node = node.right
        else:
            return True
    return False

# Find min node

def _find_min_node(self, node):
    while node and node.left:
        node = node.left
    return node

# Delete

def delete(self, key):
    def _delete(node, key):
        if not node:
            return node
        if key < node.key:
            node.left = _delete(node.left, key)
        elif key > node.key:
            node.right = _delete(node.right, key)
        else:
            # One or no child
            if not node.left:
                return node.right
            elif not node.right:
                return node.left
            # Two children
            temp = self._find_min_node(node.right)
            node.key = temp.key
            node.right = _delete(node.right, temp.key)
        return node
    self.root = _delete(self.root, key)
```

```
# Traversals

def inorder(self):
    res = [ ]
    def _in(node):
        if not node: return
        _in(node.left)
        res.append(node.key)
        _in(node.right)
    _in(self.root)
    return res

def preorder(self):
    res = [ ]
    def _pre(node):
        if not node: return
        res.append(node.key)
        _pre(node.left)
        _pre(node.right)
    _pre(self.root)
    return res

def postorder(self):
    res = [ ]
    def _post(node):
        if not node: return
        _post(node.left)
        _post(node.right)
        res.append(node.key)
    _post(self.root)
    return res

# Depth

def depth(self):
    def _depth(node):
        if not node: return 0
        return 1 + max(_depth(node.left), _depth(node.right))
    return _depth(self.root)
```

```
# Mirror

def mirror(self):
    def _mirror_node(node):
        if node is None: return None
        new_node = BSTNode(node.key)
        new_node.left = _mirror_node(node.right)
        new_node.right = _mirror_node(node.left)
        return new_node
    mirrored = BinarySearchTree()
    mirrored.root = _mirror_node(self.root)
    return mirrored

# Copy

def copy(self):
    return copy.deepcopy(self)

# Parent-child listing

def parents_with_children(self):
    pairs = []
    def _visit(node):
        if not node: return
        children = []
        if node.left: children.append(node.left.key)
        if node.right: children.append(node.right.key)
        if children:
            pairs.append((node.key, children))
        _visit(node.left)
        _visit(node.right)
    _visit(self.root)
    return pairs

# Leaf nodes

def leaf_nodes(self):
    leaves = []
    def _leaf(node):
        if not node: return
        if not node.left and not node.right:
            leaves.append(node.key)
        _leaf(node.left)
        _leaf(node.right)
    _leaf(self.root)
    return leaves
```

```
    # Level-wise

    def level_wise(self):
        levels = []
        if not self.root:
            return levels
        q = deque([self.root])
        while q:
            level_size = len(q)
            level = []
            for _ in range(level_size):
                node = q.popleft()
                level.append(node.key)
                if node.left: q.append(node.left)
                if node.right: q.append(node.right)
            levels.append(level)
        return levels

if __name__ == "__main__":
    bst = BinarySearchTree()
    while True:
        print("\n--- Binary Search Tree Menu ---")
        print("1. Insert")
        print("2. Delete")
        print("3. Search")
        print("4. Display Inorder")
        print("5. Display Preorder")
        print("6. Display Postorder")
        print("7. Display Depth")
        print("8. Display Mirror Image (Inorder)")
        print("9. Create Copy (Inorder)")
        print("10. Display Parent-Child Pairs")
        print("11. Display Leaf Nodes")
        print("12. Display Level-wise")
        print("0. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            val = int(input("Enter value to insert: "))
            bst.insert(val)
        elif choice == "2":
            val = int(input("Enter value to delete: "))
            bst.delete(val)
        elif choice == "3":
```

```
        val = int(input("Enter value to search: "))
        print("Found" if bst.search(val) else "Not found")
    elif choice == "4":
        print("Inorder:", bst.inorder())
    elif choice == "5":
        print("Preorder:", bst.preorder())
    elif choice == "6":
        print("Postorder:", bst.postorder())
    elif choice == "7":
        print("Depth of tree:", bst.depth())
    elif choice == "8":
        mirror_tree = bst.mirror()
        print("Mirror image inorder:", mirror_tree.inorder())
    elif choice == "9":
        copied_tree = bst.copy()
        print("Copied tree inorder:", copied_tree.inorder())
    elif choice == "10":
        print("Parent-Child pairs:", bst.parents_with_children())
    elif choice == "11":
        print("Leaf nodes:", bst.leaf_nodes())
    elif choice == "12":
        print("Level-wise:", bst.level_wise())
    elif choice == "0":
        print("Exiting...")
        break
    else:
        print("Invalid choice! Try again.")
```

//OUTPUT

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 1

Enter value to insert: 50

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 1

Enter value to insert: 30

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 1

Enter value to insert: 70

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 1

Enter value to insert: 20

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 1

Enter value to insert: 40

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 1

Enter value to insert: 60

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 1

Enter value to insert: 80

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 1

Enter value to insert: 70

Duplicate entry '70' not inserted.

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 2

Enter value to delete: 80

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 3

Enter value to search: 20

Found

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 4

Inorder: [20, 30, 40, 50, 60, 70]

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 5

Preorder: [50, 30, 20, 40, 70, 60]

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 6

Postorder: [20, 40, 30, 60, 70, 50]

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 7

Depth of tree: 3

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 8

Mirror image inorder: [70, 60, 50, 40, 30, 20]

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 9

Copied tree inorder: [20, 30, 40, 50, 60, 70]

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 10

Parent-Child pairs: [(50, [30, 70]), (30, [20, 40]), (70, [60])]

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 11

Leaf nodes: [20, 40, 60]

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 12

Level-wise: [[50], [30, 70], [20, 40, 60]]

--- Binary Search Tree Menu ---

1. Insert

2. Delete

3. Search

4. Display Inorder

5. Display Preorder

6. Display Postorder

7. Display Depth

8. Display Mirror Image (Inorder)

9. Create Copy (Inorder)

10. Display Parent-Child Pairs

11. Display Leaf Nodes

12. Display Level-wise

0. Exit

Enter choice: 0

Assignment No.6

Title: Represent a graph of your college campus using adjacency list /adjacency matrix. Nodes should represent the various departments/institutes and links should represent the distance between them. Find minimum spanning tree a) Using Kruskal's algorithm. b) Using Prim's algorithm.

```
class Graph:

    def __init__(self):

        self.V = 0

        self.graph = []  # For Kruskal (edge list)

        self.adj_matrix = []

        self.dept_names = []


    def add_department(self, name):

        self.dept_names.append(name)

        self.V = len(self.dept_names)

        for row in self.adj_matrix:

            row.append(0)

        self.adj_matrix.append([0] * self.V)


    def add_edge(self, u, v, w):

        self.graph.append([u, v, w])

        self.adj_matrix[u][v] = w

        self.adj_matrix[v][u] = w

  def display_adj_matrix(self):

        print("\nAdjacency Matrix:")

        for row in self.adj_matrix:

            print(row)
```

```
def display_adj_list(self):
    print("\nAdjacency List:")
    for i in range(self.V):
        connections = []
        for j in range(self.V):
            if self.adj_matrix[i][j] != 0:
                connections.append(f"{self.dept_names[j]}({self.adj_matrix[i][j]})")
        print(f"{self.dept_names[i]} -> {', '.join(connections)}")
# ---------------- Kruskal's ----------------
def find(self, parent, i):
    if parent[i] != i:
        parent[i] = self.find(parent, parent[i])
    return parent[i]
def union(self, parent, rank, x, y):
    if rank[x] < rank[y]:
        parent[x] = y
    elif rank[x] > rank[y]:
        parent[y] = x
    else:
        parent[y] = x
        rank[x] += 1
def kruskal_mst(self):
    result = []
    i = 0
    e = 0
```

```
edges = sorted(self.graph, key=lambda item: item[2])
parent = [ ]
rank = [ ]
for node in range(self.V):
    parent.append(node)
    rank.append(0)
while e < self.V - 1 and i < len(edges):
    u, v, w = edges[i]
    i += 1
    x = self.find(parent, u)
    y = self.find(parent, v)
    if x != y:
        e += 1
        result.append([u, v, w])
        self.union(parent, rank, x, y)
print("\nMST using Kruskal's Algorithm:")
total_cost = 0
for u, v, weight in result:
    print(f"{self.dept_names[u]} - {self.dept_names[v]}: {weight}")
    total_cost += weight
print("Total Distance:", total_cost)
```

```
# ---------------- Prim's ----------------
  def prim_mst(self):
      key = [float('inf')] * self.V
      parent = [-1] * self.V
      key[0] = 0
      mst_set = [False] * self.V
      for _ in range(self.V):
          min_key = float('inf')
          min_index = -1
          for v in range(self.V):
              if not mst_set[v] and key[v] < min_key:
                  min_key = key[v]
                  min_index = v
          u = min_index
          mst_set[u] = True
          for v in range(self.V):
              if self.adj_matrix[u][v] and not mst_set[v] and self.adj_matrix[u][v] < key[v]:
                  key[v] = self.adj_matrix[u][v]
                  parent[v] = u
      print("\nMST using Prim's Algorithm:")
      total_cost = 0
      for i in range(1, self.V):
          print(f"{self.dept_names[parent[i]]} - {self.dept_names[i]}: {self.adj_matrix[i][parent[i]]}")
          total_cost += self.adj_matrix[i][parent[i]]
      print("Total Distance:", total_cost)
```

```
def main():
    g = Graph()
    while True:
        print("\n--- College Campus Graph Menu ---")
        print("1. Add Department")
        print("2. Add Distance Between Departments")
        print("3. Display Adjacency Matrix")
        print("4. Display Adjacency List")
        print("5. Find MST (Kruskal's Algorithm)")
        print("6. Find MST (Prim's Algorithm)")
        print("7. Exit")
        choice = input("Enter your choice: ")

        if choice == '1':
            name = input("Enter Department Name: ")
            g.add_department(name)
        elif choice == '2':
            if g.V < 2:
                print("Add at least two departments first!")
                continue
            print("\nDepartments:")
            for idx, dept in enumerate(g.dept_names):
                print(f"{idx}: {dept}")
            u = int(input("Enter index of Department 1: "))
            v = int(input("Enter index of Department 2: "))
```

```
            w = int(input("Enter Distance: "))
            g.add_edge(u, v, w)

        elif choice == '3':
            g.display_adj_matrix()

        elif choice == '4':
            g.display_adj_list()

        elif choice == '5':
            g.kruskal_mst()

        elif choice == '6':
            g.prim_mst()

        elif choice == '7':
            print("Exiting...")
            break
        else:
            print("Invalid choice, try again.")

if __name__ == "__main__":
    main()
```

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 1

Enter Department Name: IT

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 1

Enter Department Name: mech

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 1

Enter Department Name: civil

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 1

Enter Department Name: etc

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 2

Departments:

0: IT

1: mech

2: civil

3: etc

Enter index of Department 1: 0

Enter index of Department 2: 1

Enter Distance: 5

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 2

Departments:

0: IT

1: mech

2: civil

3: etc

Enter index of Department 1: 1

Enter index of Department 2: 2

Enter Distance: 7

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 2

Departments:

0: IT

1: mech

2: civil

3: etc

Enter index of Department 1: 2

Enter index of Department 2: 3

Enter Distance: 9

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 3

Adjacency Matrix:

[0, 5, 0, 0]

[5, 0, 7, 0]

[0, 7, 0, 9]

[0, 0, 9, 0]

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 4

Adjacency List:

IT -> mech(5)

mech -> IT(5), civil(7)

civil -> mech(7), etc(9)

etc -> civil(9)

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 5

MST using Kruskal's Algorithm:

IT - mech: 5

mech - civil: 7

civil - etc: 9

Total Distance: 21

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 6

MST using Prim's Algorithm:

IT - mech: 5

mech - civil: 7

civil - etc: 9

Total Distance: 21

--- College Campus Graph Menu ---

1. Add Department

2. Add Distance Between Departments

3. Display Adjacency Matrix

4. Display Adjacency List

5. Find MST (Kruskal's Algorithm)

6. Find MST (Prim's Algorithm)

7. Exit

Enter your choice: 7

Exiting...

Assignment No.7

Title: Design and implement the Heap Sort algorithm to efficiently sort an array of integers in ascending order. The implementation should be optimized for time and space complexity and should clearly demonstrate the working principles of heap data structures (min-heap or max-heap as applicable)

```
def heapify(arr, n, i):
    """Ensures the subtree rooted at index i is a max-heap."""
    largest = i
    left = 2 * i + 1     # left child index
    right = 2 * i + 2    # right child index

    # Check if left child exists and is greater than root
    if left < n and arr[left] > arr[largest]:
        largest = left

    # Check if right child exists and is greater than current largest
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If largest is not the root, swap and continue heapifying
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
def heap_sort(arr):
    n = len(arr)
    # Build max-heap
    for i in range(n // 2 - 1, -1, -1):
```

```
        heapify(arr, n, i)
    print("\nMax Heap constructed:", arr)
    # Extract elements from heap one by one
    for i in range(n - 1, 0, -1):
        # Swap root (largest) with the last element
        arr[0], arr[i] = arr[i], arr[0]
        # Heapify the reduced heap
        heapify(arr, i, 0)
        print(f"Heap after removing element {n-i}:", arr)
def main():
    while True:
        print("\n--- Heap Sort Menu ---")
        print("1. Enter array")
        print("2. Perform Heap Sort")
        print("3. Exit")
        choice = input("Enter choice: ")
        if choice == "1":
            global arr
            arr = list(map(int, input("Enter integers separated by space: ").split()))
        elif choice == "2":
            if not arr:
                print("Array is empty! Please enter array first.")
            else:
                print("\nOriginal Array:", arr)
                heap_sort(arr)
```

```
            print("\nSorted Array (Ascending):", arr)
        elif choice == "3":
            print("Exiting program...")
            break
        else:
            print("Invalid choice! Please try again.")


if __name__ == "__main__":
    arr = []
    main()
```

//OUTPUT

```
--- Heap Sort Menu ---
1. Enter array
2. Perform Heap Sort
3. Exit
Enter choice: 1
Enter integers separated by space: 1 2 3 4 5 6 7 8
--- Heap Sort Menu ---
1. Enter array
2. Perform Heap Sort
3. Exit
Enter choice: 2
Original Array: [1, 2, 3, 4, 5, 6, 7, 8]
Max Heap constructed: [8, 5, 7, 4, 1, 6, 3, 2]
Heap after removing element 1: [7, 5, 6, 4, 1, 2, 3, 8]
```

Heap after removing element 2: [6, 5, 3, 4, 1, 2, 7, 8]

Heap after removing element 3: [5, 4, 3, 2, 1, 6, 7, 8]

Heap after removing element 4: [4, 2, 3, 1, 5, 6, 7, 8]

Heap after removing element 5: [3, 2, 1, 4, 5, 6, 7, 8]

Heap after removing element 6: [2, 1, 3, 4, 5, 6, 7, 8]

Heap after removing element 7: [1, 2, 3, 4, 5, 6, 7, 8]

Sorted Array (Ascending): [1, 2, 3, 4, 5, 6, 7, 8]

--- Heap Sort Menu ---

1. Enter array

2. Perform Heap Sort

3. Exit

Enter choice:

Assignment No.8

Title: Sort Online Orders by Delivery Time using Merge Sort. Problem Statement: You are given a list of online orders, each with an estimated delivery time in minutes. Write a program to sort these orders using the Merge Sort algorithm so the delivery system can prioritize quicker deliveries first.

```
def merge(arr, left, mid, right):
    """Merge two sorted halves of arr."""
    # Create temporary subarrays
    L = arr[left:mid+1]
    R = arr[mid+1:right+1]
    i = j = 0
    k = left
    # Merge while comparing delivery times
    while i < len(L) and j < len(R):
        if L[i][1] <= R[j][1]:  # Compare delivery times
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1
    # Copy remaining elements
    while i < len(L):
        arr[k] = L[i]
        i += 1
        k += 1
```

```
    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1
def merge_sort(arr, left, right):
    """Recursive merge sort."""
    if left < right:
        mid = (left + right) // 2
        merge_sort(arr, left, mid)
        merge_sort(arr, mid+1, right)
        merge(arr, left, mid, right)
def display_orders(orders):
    """Pretty print the orders."""
    print("\nOrders:")
    print("{:<5} {:<20} {:<15}".format("ID", "Item", "Delivery Time (min)"))
    for order in orders:
        print("{:<5} {:<20} {:<15}".format(order[0], order[1], order[2]))
def main():
    orders = []
    while True:
        print("\n--- Online Orders Merge Sort Menu ---")
        print("1. Add Order")
        print("2. Display Orders")
        print("3. Sort Orders by Delivery Time")
        print("4. Exit")
```

```
choice = input("Enter choice: ")
if choice == "1":
    order_id = input("Enter Order ID: ")
    item_name = input("Enter Item Name: ")
    delivery_time = int(input("Enter Delivery Time (in minutes): "))
    orders.append((order_id, item_name, delivery_time))
elif choice == "2":
    if orders:
        display_orders(orders)
    else:
        print("No orders available!")
elif choice == "3":
    if orders:
        merge_sort(orders, 0, len(orders)-1)
        print("\nOrders sorted by delivery time:")
        display_orders(orders)
    else:
        print("No orders to sort!")
elif choice == "4":
    print("Exiting program...")
    break
else:
    print("Invalid choice! Please try again.")
```

```
if __name__ == "__main__":
    main()
//output
--- Online Orders Merge Sort Menu ---
1. Add Order
2. Display Orders
3. Sort Orders by Delivery Time
4. Exit
Enter choice: 1
Enter Order ID: 101
Enter Item Name: laptop
Enter Delivery Time (in minutes): 40
--- Online Orders Merge Sort Menu ---
1. Add Order
2. Display Orders
3. Sort Orders by Delivery Time
4. Exit
Enter choice: 1
Enter Order ID: 102
Enter Item Name: mobile
Enter Delivery Time (in minutes): 45
--- Online Orders Merge Sort Menu ---
1. Add Order
2. Display Orders
3. Sort Orders by Delivery Time
```

4. Exit

Enter choice: 1

Enter Order ID: 103

Enter Item Name: book

Enter Delivery Time (in minutes): 50

--- Online Orders Merge Sort Menu ---

1. Add Order

2. Display Orders

3. Sort Orders by Delivery Time

4. Exit

Enter choice: 2

Orders:

ID	Item	Delivery Time (min)
101	laptop	40
102	mobile	45
103	book	50

--- Online Orders Merge Sort Menu ---

1. Add Order

2. Display Orders

3. Sort Orders by Delivery Time

4. Exit

Enter choice: 3

Orders sorted by delivery time:

Orders:

ID	Item	Delivery Time (min)

```
103   book        50
101   laptop      40
102   mobile       45
--- Online Orders Merge Sort Menu ---
1. Add Order
2. Display Orders
3. Sort Orders by Delivery Time
4. Exit
Enter choice:
```

Assignment No.9

Title: Maximize Profit by Shipping Partial Orders (Fractional Knapsack) Problem Statement: You run a shipping company and need to load a truck with parcels of different weights and profits. The truck has a limited weight capacity. Write a program to choose parcels (even partially) to maximize profit using the Fractional Knapsack strategy.

```
def fractional_knapsack(weights, profits, capacity):
    ratio = [(profits[i] / weights[i], weights[i], profits[i]) for i in range(len(weights))]
    ratio.sort(reverse=True, key=lambda x: x[0])
    total_profit = 0
    selected_items = []
    for r, w, p in ratio:
        if capacity <= 0:
            break
        if w <= capacity:
            capacity -= w
            total_profit += p
            selected_items.append((w, p, 1.0))
        else:
            fraction = capacity / w
            total_profit += p * fraction
            selected_items.append((w, p, fraction))
            capacity = 0
    return total_profit, selected_items
```

```
def input_data():
    n = int(input("Enter number of parcels: "))
    weights = []
    profits = []
    for i in range(n):
        w = float(input(f"Enter weight of parcel {i+1}: "))
        p = float(input(f"Enter profit of parcel {i+1}: "))
        weights.append(w)
        profits.append(p)
    capacity = float(input("Enter truck capacity: "))
    return weights, profits, capacity
def display_result(selected, max_profit):
    print("\nSelected parcels:")
    for w, p, frac in selected:
        print(f"Weight: {w}, Profit: {p}, Taken: {frac*100:.2f}%")
    print(f"\nMaximum Profit: {max_profit:.2f}\n")

# Main menu loop
while True:
    print("\n=== Fractional Knapsack Menu ===")
    print("1. Enter Data and Calculate")
    print("2. Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        weights, profits, capacity = input_data()
```

```
        max_profit, selected = fractional_knapsack(weights, profits, capacity)
        display_result(selected, max_profit)
    elif choice == "2":
        print("Exiting... Goodbye!")
        break
    else:
        print("Invalid choice. Try again.")
```

//OUTPUT

```
1. Enter Data and Calculate
2. Exit
Enter choice: 1
Enter number of parcels: 3
Enter weight of parcel 1: 10
Enter profit of parcel 1: 60
Enter weight of parcel 2: 20
Enter profit of parcel 2: 100
Enter weight of parcel 3: 30
Enter profit of parcel 3: 120
Enter truck capacity: 50
Selected parcels:
Weight: 10.0, Profit: 60.0, Taken: 100.00%
Weight: 20.0, Profit: 100.0, Taken: 100.00%
Weight: 30.0, Profit: 120.0, Taken: 66.67%
Maximum Profit: 240.00
```

Assignment No.10

Title: Given: l A text string text of length n. l A pattern string pattern of length m. Objective: Find all starting indices i in the text such that the substring text[i:i+m] is exactly equal to the pattern pattern, using the Naive String Matching Algorithm approach. Constraints: l 0 ≤ m ≤ n l Characters in text and pattern can be any valid characters (e.g., a–z, A–Z, digits, etc.)

```
# Menu-Driven Naive String Matching Algorithm


def naive_string_match(text, pattern):
    n = len(text)
    m = len(pattern)
    positions = [ ]


    for i in range(n - m + 1):
        if text[i:i+m] == pattern:
            positions.append(i)
    return positions


def main():
    text = ""
    pattern = ""
    while True:
        print("\n===== Naive String Matching Menu =====")
        print("1. Enter Text String")
        print("2. Enter Pattern String")
        print("3. Search Pattern in Text")
        print("4. Exit")
```

```
        choice = input("Enter your choice: ")
        if choice == "1":
            text = input("Enter the text string: ")
        elif choice == "2":
            pattern = input("Enter the pattern string: ")
        elif choice == "3":
            if len(pattern) > len(text):
                print("Error: Pattern length cannot be greater than text length.")
            elif text == "" or pattern == "":
                print("Please enter both text and pattern first.")
            else:
                matches = naive_string_match(text, pattern)
                if matches:
                    print(f"Pattern found at starting indices: {matches}")
                else:
                    print("No match found.")
        elif choice == "4":
            print("Exiting program...")
            break
        else:
            print("Invalid choice! Please try again.")
if __name__ == "__main__":
    main()
```

//OUTPUT

===== Naive String Matching Menu =====

1. Enter Text String

2. Enter Pattern String

3. Search Pattern in Text

4. Exit

Enter your choice: 1

Enter the text string: data structure and algorithm

===== Naive String Matching Menu =====

1. Enter Text String

2. Enter Pattern String

3. Search Pattern in Text

4. Exit

Enter your choice: 2

Enter the pattern string: algo

===== Naive String Matching Menu =====

1. Enter Text String

2. Enter Pattern String

3. Search Pattern in Text

4. Exit

Enter your choice: 3

Pattern found at starting indices: [19]

===== Naive String Matching Menu =====

1. Enter Text String

2. Enter Pattern String

3. Search Pattern in Text

4. Exit

Enter your choice: